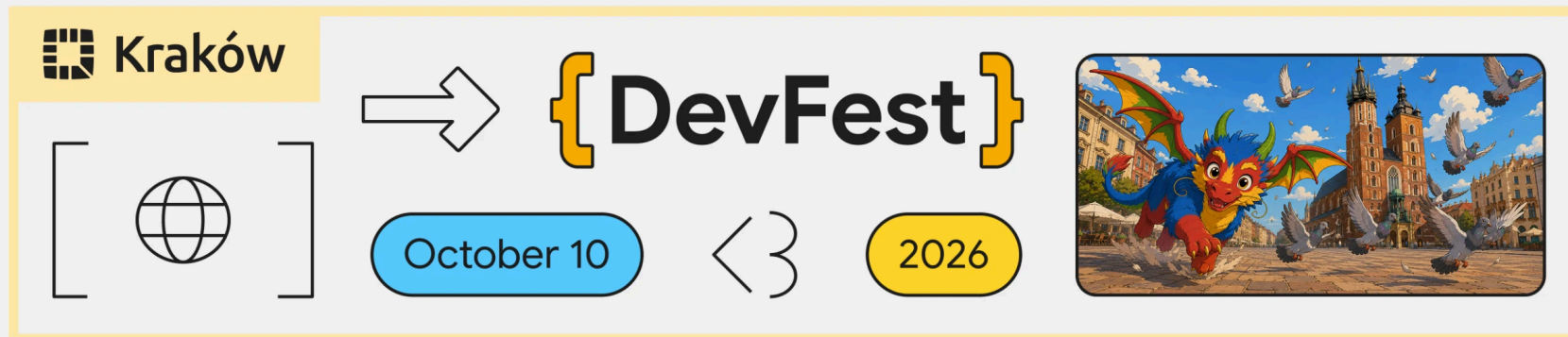




 Google Developer Experts • DevFest Kraków 2026

Building a Local-First Cultural AI Assistant

Full-Stack Masterclass with Google ADK, Gemma 4, and In-Memory Dual-Layer RAG

Speaker: Nuno Andrade • Google Developer Expert (GDE) | **Event:** DevFest Kraków 2026

Runtime: 100% On-Device Node.js LTS | Local Gemma 4 via Ollama | Zero Vector DB Bloat



Speaker Introduction

• Nuno Andrade • Google Developer Expert

AI/ML & Web Technologies

Passionate about on-device machine learning, agentic architectures, and building lean, resilient software systems without unnecessary cloud dependencies.

- Active GDG & GDE community mentor
- Production Node.js & Distributed Systems architect
- Advocate for privacy-preserving, local-first AI
- **LinkedIn:** [linkedin.com/in/nuno-andrade-964b2416](https://www.linkedin.com/in/nuno-andrade-964b2416)

DevFest Kraków 2026 Masterclass

Today's Goal: Transition from toy AI API wrappers to robust, production-ready AI agents using Google's official Agent Development Kit (ADK).

Core Philosophy:

- **Local First:** Privacy, zero API bills, full offline support.
- **Zero DB Bloat:** In-memory lexical search beats vector DBs for domain tasks.
- **Official Standards:** Google ADK component lifecycle.

The AI Agent Paradigm Shift

The Cloud AI Anti-Pattern

- **Unpredictable Costs:** Metred per-token API bills that explode under load.
- **Privacy Risk:** Sending sensitive user queries and city data to third-party servers.
- **Latency Tax:** 300ms-800ms pure HTTP roundtrip before token generation begins.
- **Brittle Infrastructure:** Requires Dockerized vector DBs, Python daemons, and microservices.

The Local-First Modern Standard

- **100% On-Device:** Local model (Gemma 4 `gemma4:e2b` or Gemma 2 `gemma2:2b` via `OLLAMA_MODEL`) runs on user hardware via Ollama.
- **Zero External Vector DB:** In-memory BM25 retrieval finishes in **< 2 milliseconds**.
- **Zero-Downtime Hot Reload:** Edit markdown files live without server restarts.
- **Unified Node.js ESM:** Google ADK Agent + Express SPA in a single process.

Workshop Architecture Blueprint

Single Node.js Process Architecture: Concurrently serves the user-facing web app and official developer tooling while sharing agent state, models, and RAG cache.

FRONTEND & API

Port 3030: Express 5

- Vanilla HTML5/CSS3 SPA
- Zero build tools (no Vite)
- Grounding citations display
- Tool execution badges

ORCHESTRATION

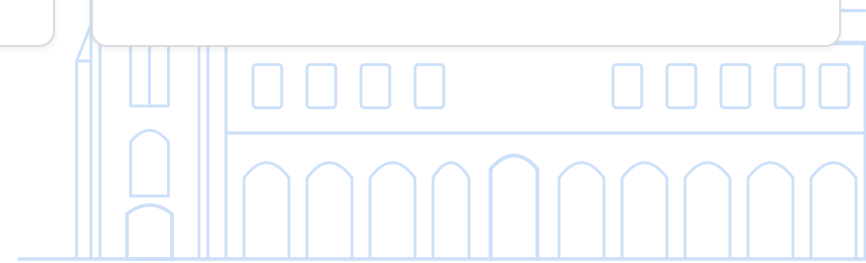
Google ADK Agent

- `Agent` & `InMemoryRunner`
- Custom `OllamaLlm` (BaseLlm)
- Native `FunctionTool` wraps
- Offline Direct Mode fallback

EVALUATION

Port 8000: ADK Dev-UI

- Official `AdkApiServer`
- Visual agent graph explorer
- Step-by-step trace inspection
- Tool payload debugger



6-Step Progressive Curriculum Roadmap

Step	Milestone	Tech & Capabilities	Automated Tests
01	Setup & Local LLM	Ollama daemon, Gemma 4 / Gemma 2 via <code>OLLAMA_MODEL</code> , Node.js client	5 tests
02	Dual-Layer RAG Engine	In-memory BM25, Polish diacritics, markdown chunking, <code>mtime</code> hot-reload	9 tests
03	Native Tool Binding	JSON Schema declarations, Hejnał time math, Wawel ticket simulator	7 tests
04	Google ADK Orchestration	<code>@google/adk</code> , <code>BaseLlm</code> , <code>FunctionTool</code> , <code>InMemoryRunner</code> tool loops	23 tests
05	Full Application & UI	Unified Express SPA (:3030) + Google ADK Web Dev-UI (:8000) dual listener	35 tests
06	Google Search MCP	Model Context Protocol studio server, live search & dynamic cross-content	42 tests
TOTAL	All 6 Steps Combined	Full production local cultural concierge application	121 tests

Workshop Logistics & Repository Navigation

Self-Contained Step Folders

```
steps/  
├── step-01-setup-and-llm/  
├── step-02-dual-layer-rag/  
├── step-03-native-tools/  
├── step-04-adk-agent/  
├── step-05-full-app/  
└── step-06-google-search-mcp/
```

Each directory contains its own `package.json`, source code, automated test suite, and exhaustive README.

Global Workspace Commands

```
# Install all dependencies  
npm install
```

Run all 121 tests across 6 steps

```
npm test
```

Run tests for a specific step

```
npm run test:step-01  
npm run test:step-06
```

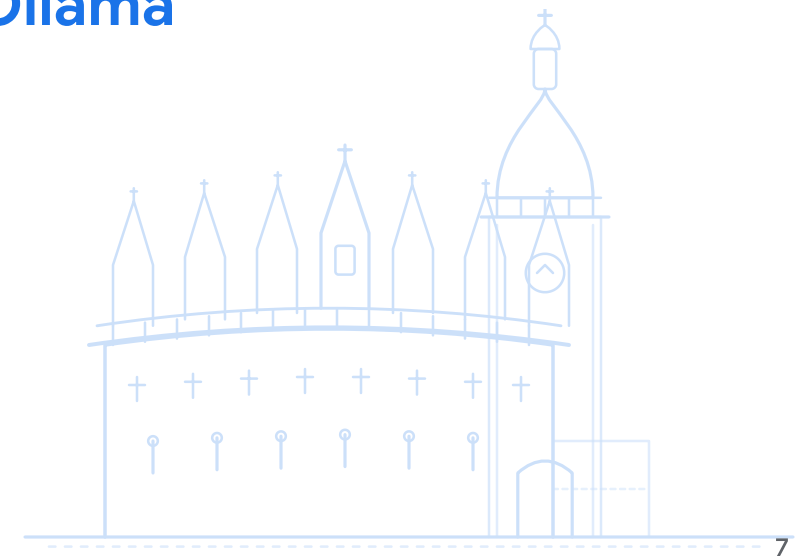
Launch the unified application

```
npm start
```

• Step 01

Step 01: Setup & Local LLM Connectivity

Harnessing Google's Gemma 4 on Local Hardware via Ollama



Prerequisites: Install Node.js & Ollama

Quick Installation Guide for Linux & macOS

1. NODE.JS LTS VIA NVM

Node Version Manager (Linux & macOS)

```
# 1. Install NVM
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
# 2. Reload shell configuration
source ~/.nvm/nvm.sh
# 3. Install & use Node.js 22 LTS
nvm install 22
nvm use 22
node -v    # Expected: v22.x.x
npm -v     # Expected: 10.x.x+
```

2. LOCAL OLLAMA DAEMON

Ollama Installation & Startup

Linux Installation:

```
curl -fsSL https://ollama.com/ins
```

macOS Installation:

```
brew install ollama
```

Start Daemon & Pull Model:

```
ollama serve                # Start
ollama pull gemma4:e2b      # Defau
# Fallback: ollama pull gemma2:2b
```

Why Gemma & Ollama?

High-Speed Inference

Runs locally via Ollama with fast CPU/GPU inference (~1.8 GB VRAM for `gemma4:e2b` or `gemma2:2b`).

Dynamic Configuration

Zero hardcoded models! Configured via `OLLAMA_MODEL` in `.env` with instant fallback.

Instruction-Tuned

Fine-tuned for structured instruction following, JSON output generation, and seamless tool call decisions.

Verify Local Setup in Terminal:

```
ollama serve          # Ensure daemon is running
ollama pull gemma4:e2b # Default model (~1.8 GB)
# Or for lower RAM: ollama pull gemma2:2b
export OLLAMA_MODEL=gemma4:e2b
```



Step 01: Node.js Connectivity (src/llm.js)

1. Config & Health Check

```
export const OLLAMA_MODEL =
  process.env.OLLAMA_MODEL ||
  process.env.MODEL || 'gemma4:e2b';
export async function
checkOllamaHealth(
  client = createOllamaClient(),
  targetModel = OLLAMA_MODEL
) {
  const res = await client.list();
  const names = res.models.map(m =>
m.name);
  return {
    online: true,
    modelFound:
names.includes(targetModel)
  };
}
```

2. Deterministic Inference

```
export async function queryGemma(prompt, opts) {
  const client = opts.client || createOllamaClient();
  const model = opts.model || OLLAMA_MODEL;
  const res = await client.generate({
    model,
    prompt,
    stream: false,
    options: {
      temperature: 0.2 // Low temp for facts!
    }
  });
  return {
    response: res.response.trim(),
    durationMs: Date.now() - start
  };
}
```

Step 01: Hands-On Verification

Run Step 1 Verification Script

```
cd steps/step-01-setup-and-llm
cp .env.example .env
npm start
```

Expected Console Output:

```
Checking local Ollama connectivity at: http://127.0.0.1:11434
Target Model: gemma4:e2b
Ollama daemon is ONLINE.
Available models: gemma4:e2b
Model "gemma4:e2b" is ready locally!
Sending test query to Gemma 4: "Summarize Kraków in one short sentence."
Gemma 4 Response: Kraków is a historic Polish city renowned for its preserved medieval
Old Town, royal Wawel Castle, and vibrant cultural heritage.
Duration: 1173ms
```

5 AUTOMATED TESTS PASSING

Run `npm test` to verify unit test coverage.

● Step 02

Step 02: Dual-Layer RAG Engine

Eliminating Vector Database Bloat with In-Memory Lexical Retrieval



Why Vector Databases are Overkill for Local RAG

The Vector DB Trap

- **Scale Mismatch:** Built for billions of vectors; domain knowledge is ~50-500 paragraphs (< 2 MB).
- **VRAM Tax:** Embedding model steals 1.5 GB VRAM from the generative LLM.
- **Double Latency:** Generating query embeddings adds 50ms-200ms overhead.
- **Semantic Drift:** Dense vectors confuse numbers, prices, and specific codes.

Our In-Memory Lexical Engine

- **Zero Dependencies:** 100% native Node.js standard library.
- **Sub-Millisecond Speed:** BM25 RAM traversal finishes in < 2 ms.
- **Exact Precision:** BM25 rewards exact keyword matches (e.g. "Dragon's Den PLN").
- **Instant Hot Reload:** Filesystem `mtime` cache updates automatically.



The Dual-Layer Knowledge Architecture

LAYER 1: STATIC CORPUS

Immutable Historical Knowledge

Compiled historical facts that rarely change:

- **Wawel Royal Hill:** Casimir the Great, Renaissance courtyard, Smok Wawelski dragon legend.
- **St. Mary's Basilica:** Veit Stoss carved wood altarpiece, asymmetric towers, 1241 Mongol siege.
- **Main Market Square:** 1257 Magdeburg charter layout, Sukiennice Cloth Hall.
- **Kazimierz:** 1335 royal charter, Old Synagogue, Schindler's Factory.

LAYER 2: MUTABLE DYNAMIC

Live Hot-Reloading Markdown

Frequently changing operational data in `data/mutable/`:

- `wawel_pricing.md`: Admission prices, free Monday policies, ticket contacts.
- `tourist_services.md`: Emergency lines (112, 997, 986), InfoKraków visitor centers, luggage storage.
- **Zero Downtime:** Edit any `.md` file on disk; the engine updates in RAM on the next request!

Polish Diacritics & BM25 Scoring

Diacritic Normalization

```
export function tokenize(text) {  
  if (!text || typeof text !== 'string') return [];  
  return text  
    .toLowerCase()  
    .normalize('NFD') // Decompose accents  
    .replace(/[\u0300-\u036f]/g, '') // Strip marks  
    .replace(/^[^w\s]/g, ' ')  
    .split(/\s+/)  
    .filter(t => t.length > 2 && !STOP_WORDS.has(t));  
}
```

Normalizes Polish letters (**ą→a** , **ć→c** , **ł→l** , **ź→z**) so searches work regardless of keyboard layout.

Filesystem **mtime** Invalidation

```
// In DualLocalRAGEngine  
const stats = fs.statSync(filePath);  
if (this.cache[file]?.mtime === stats.mtimeMs) {  
  // Cache Hit: sub-millisecond RAM read!  
  chunks.push(...this.cache[file].chunks);  
  continue;  
}  
// Cache Miss: re-chunk markdown heading sections  
const content = fs.readFileSync(filePath, 'utf8');  
const fresh = chunkMarkdown(content, file);  
this.cache[file] = { mtime: stats.mtimeMs, chunks: fresh };
```

Step 02: Hands-On Live Hot-Reload Lab

1. Run Unit Tests

```
cd steps/step-02-dual-layer-rag  
npm test
```

9 TESTS PASS IN ~20MS

- Tokenization & diacritic preservation
- Heading-based markdown chunking
- Static & mutable query retrieval
- Dynamic hot-reloading verification

2. Try Live Hot-Reloading!

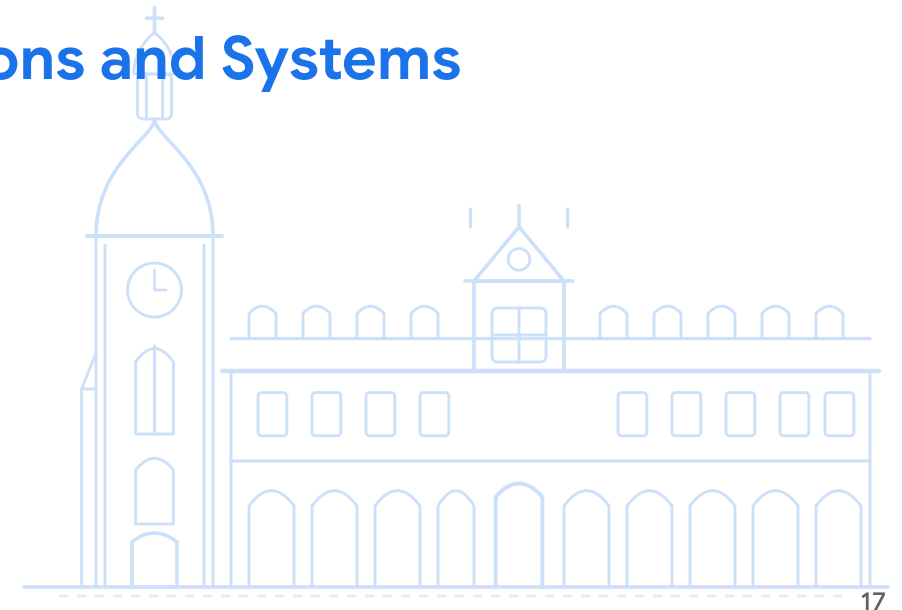
1. Open `data/mutable/wawel_pricing.md`.
2. Change State Rooms price from `35 PLN` to `55 PLN`.
3. Save the file.
4. Re-run test or query: the engine immediately retrieves the updated `55 PLN` price!

Zero downtime. Zero re-indexing pipeline.

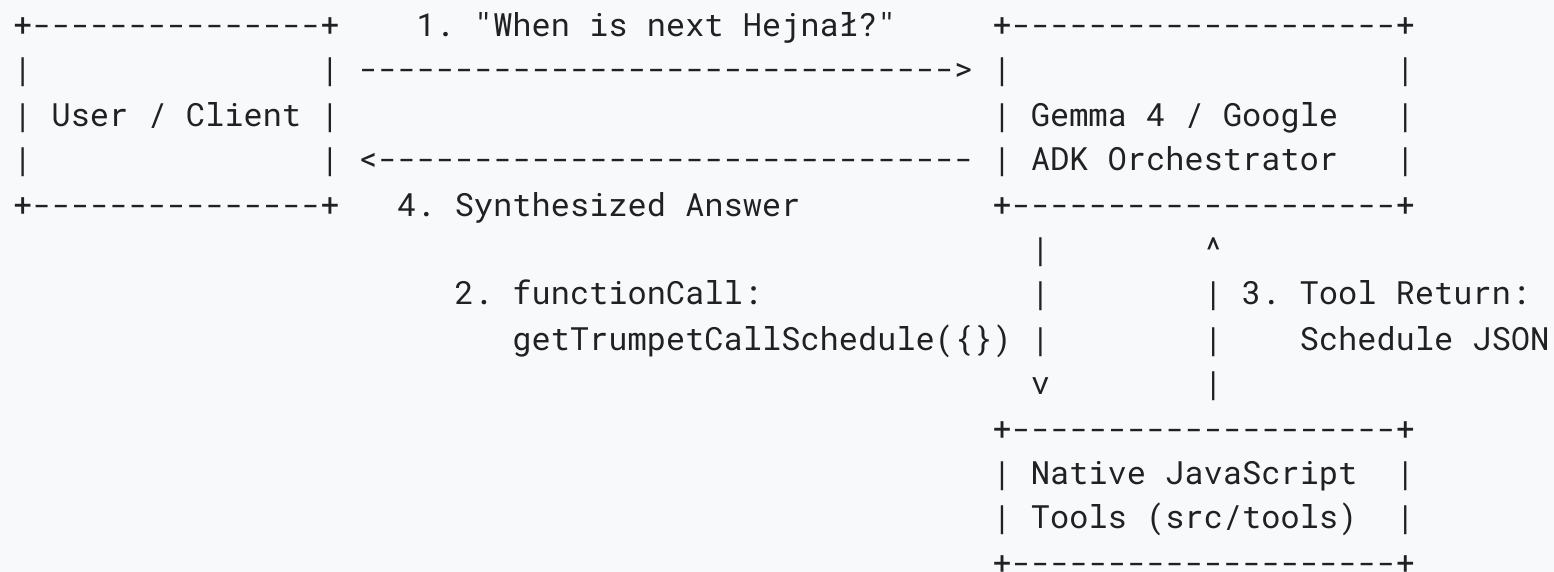
● Step 03

Step 03: Native Tool Binding & Execution

Grounding Language Models in Real-Time Calculations and Systems



The LLM Function Calling Protocol



Why Tools are Vital: LLMs cannot tell the current time, compute modulo arithmetic reliably, or know real-time museum ticket quotas. Tools give the agent *hands and eyes*.

The Three Native Domain Tools

TOOL 1: SCHEDULE MATH

getTrumpetCallSchedule

- Computes minutes remaining until the next hourly *Hejnał Mariacki*.
- Returns 4 cardinal dedications: South (King), West (Mayor), North (Guard), East (Fire Chief).
- Details the 1241 Mongol siege acoustic cutoff.

TOOL 2: INVENTORY SIMULATOR

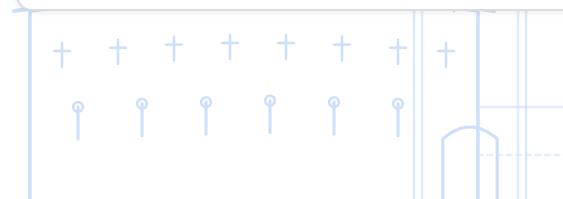
getWawelTicketAvailability

- Simulates live capacity across State Rooms, Royal Crypts, and Dragon's Den.
- Handles dates, defaults to today.
- Enforces Monday free-admission rules.
- Flags sold-out alerts.

TOOL 3: CURATED GUIDE

recommendLocalDining

- Filters by district: **Old Town**, **Kazimierz**, **Podgórze**.
- Filters by budget tier: **budget / milk bar**, **moderate**, **fine dining**.
- Includes Bar Mleczny Górnik and 2-star Bottiglieria 1881.



Declaring Tools with JSON Schema

```
export const toolDefinitions = [
  {
    type: 'function',
    function: {
      name: 'getWawelTicketAvailability',
      description: 'Check real-time ticket availability and remaining capacity for Wawel Castle exhibitions.',
      parameters: {
        type: 'object',
        properties: {
          date: {
            type: 'string',
            description: 'Visit date in YYYY-MM-DD format. Defaults to today if omitted.'
          }
        },
        required: []
      }
    }
  }
];
```

- **Self-Describing Contracts:** The LLM reads the `description` to determine **intent**.
- **Runtime Dispatch:** Dynamic execution via registry: `await toolsByName[toolName](args)`.

Step 03: Hands-On Tool Verification

Execute Tool Unit Tests

```
cd steps/step-03-native-tools  
npm test
```

7 AUTOMATED TESTS PASSING

- Real-time minute calculation math
- Cardinal direction dedications
- Museum exhibition capacity simulation
- District & budget dining filter logic
- JSON Schema export validity

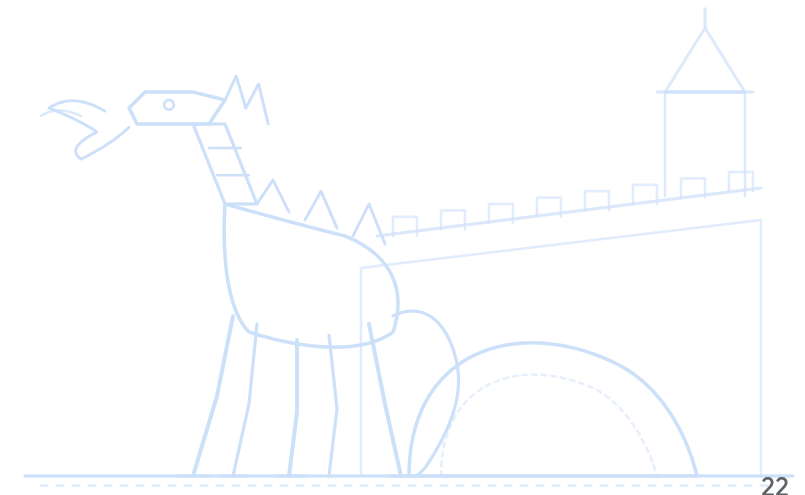
Direct Execution in Node REPL

```
// Direct invocation without LLM  
import { getTrumpetCallSchedule }  
  from './src/tools.js';  
const res = await  
getTrumpetCallSchedule();  
console.log(res);  
// {  
//   nextOccurrenceInMinutes: 34,  
//   nextScheduledTime: "11:00",  
//   cardinalDirections: [...]  
// }
```

● Step 04

Step 04: Google ADK Orchestration

Assembling Autonomous Multi-Turn Agent Loops with **@google/adk**



Official Google ADK Component Architecture

The Google ADK Object Model

- **Agent** : High-level container encapsulating name, instruction, model instance, and registered tools.
- **BaseLlm** : Abstract base class for language models. Defines `async *generateContentAsync()` generator contract.
- **FunctionTool** : Typed tool wrapper that validates input payloads and executes native JS functions.
- **InMemoryRunner** : Coordinates conversational turns and drives autonomous tool-execution loops.

Why Google ADK?

- Official Google framework for agentic workflows in Node.js and TypeScript.
- Standardized streaming and telemetry abstractions.
- Full compatibility with Google developer ecosystem and DevTools.
- Decoupled from specific model vendors (supports Gemini, Gemma, custom LLMs).

Bridging Ollama & ADK: Custom **OllamaLlm**

```
import { BaseLlm } from '@google/adk';

export class OllamaLlm extends BaseLlm {
  constructor({
    model = process.env.OLLAMA_MODEL || process.env.MODEL || 'gemma4:e2b',
    host = process.env.OLLAMA_HOST || 'http://127.0.0.1:11434'
  } = {}) {
    super({ model });
    this.client = new Ollama({ host });
  }

  async *generateContentAsync(llmRequest, stream, abortSignal) {
    const messages = transformAdkContentsToOllama(llmRequest.contents);
    const tools = formatToolsForOllama(llmRequest.tools);

    const response = await this.ollama.chat({ model: this.model, messages, tools });

    yield {
      content: {
        role: 'model',
        parts: response.message.tool_calls
          ? response.message.tool_calls.map(tc => ({
              functionCall: { id: tc.id, name: tc.function.name, args: tc.function.arguments }
            }))
          : [{ text: response.message.content }],
      },
      finishReason: 'STOP',
      turnComplete: true
    };
  }
}
```

Grounded Execution & Autonomous Tool Loop

```
sequenceDiagram
    participant User
    participant Runner as InMemoryRunner
    participant Agent as Google ADK Agent
    participant RAG as DualLocalRAGEngine
    participant LLM as OllamaLlm
    participant Tool as FunctionTool

    User->>Runner: runEphemeral(agent, "When is next Hejnał?")
    Runner->>RAG: query("When is next Hejnał?")
    RAG-->>Runner: Grounded context & citations
    Runner->>LLM: generateContentAsync(groundedPrompt)
    LLM-->>Runner: Yields functionCall: getTrumpetCallSchedule
    Note over Runner: ADK Autonomous Loop
    Runner->>Tool: execute({})
    Tool-->>Runner: Return schedule JSON
    Runner->>LLM: generateContentAsync(context + toolResult)
    LLM-->>Runner: Yields text: "The next Hejnał is in 14 minutes..."
    Runner-->>User: Complete synthesized response
```

Step 04: Hands-On Verification & Fallback

Run Step 4 Test Suite

```
cd steps/step-04-adk-agent  
cp .env.example .env  
npm test
```

23 TESTS PASSING ACROSS 3 SUITES

- ADK Agent instantiation & prompt grounding
- FunctionTool argument validation
- End-to-end multi-turn tool loops
- Offline fallback resilience

Resilient Offline Fallback Mode

What if the Ollama daemon crashes or is restarting?

Grounded RAG Direct Mode:

The agent detects daemon unavailability, dispatches relevant tools deterministically, retrieves RAG chunks, and crafts a grounded response without crashing.

• Step 05

Step 05: Full Production Application & UI

Unified Express 5 SPA & Official Google ADK Web Dev-UI



Unified Dual-Port Server Architecture

Single Node.js Process (`src/server.js`): Both servers run concurrently, sharing memory, RAG indexes, and the agent instance.

PORT 3030

Express 5 REST API & SPA

- `POST /api/chat` : Conversational endpoint
- `GET /api/health` : System & RAG telemetry
- `GET /api/tools` : JSON Schema list
- `POST /api/tools/:name` : Direct tool runner
- `GET /dev-ui` : Redirects to port 8000
- Zero build step: vanilla SPA delivery

PORT 8000

Google ADK Web Dev-UI

- Powered by `@google/adk-devtools`
- Interactive agent component graph
- Turn-by-turn trace visualizer
- Live state & memory inspector
- Full support for `npx adk web`

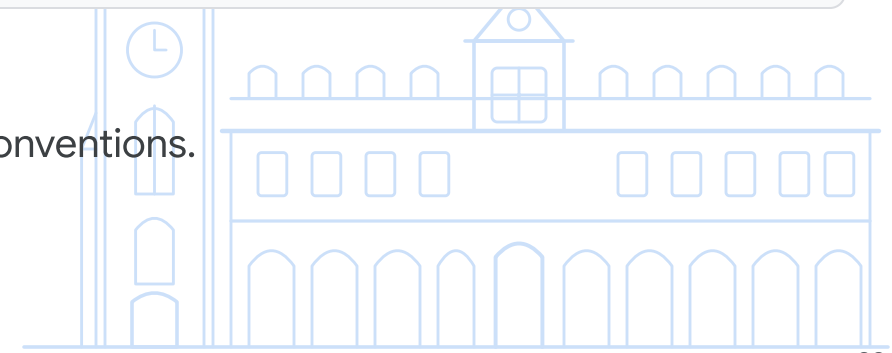
Pure ESM Root Agent Export for ADK CLI

To allow Google ADK's native CLI (`npx adk web`) to discover and inspect our agent:

```
// krakow_cultural_agent/agent.js
import { Agent } from '@google/adk';
import { OllamaLlm } from '../src/agent.js';
import { getFunctionTools } from '../src/agent.js';

export const rootAgent = new Agent({
  name: 'krakow_cultural_agent',
  description: 'AI Tour Guide and Concierge for the Royal Capital City of Kraków',
  model: new OllamaLlm(),
  tools: getFunctionTools(),
  instruction: 'You are the Kraków Cultural AI Tour Guide and Concierge...'
});
```

- **Zero Python Dependencies:** Pure JavaScript ESM module.
- **Standardized Contract:** Exports `rootAgent` as mandated by Google ADK conventions.



Step 05: Hands-On Full App Launch

Launch Unified Server

```
cd steps/step-05-full-app  
cp .env.example .env  
npm start
```

1. Kraków Tour Concierge SPA

Open browser to: **<http://localhost:3030>**

- Try quick prompt chips
- Observe tool execution badges
- View grounding source attribution

2. Google ADK Web Dev-UI

Open browser to: **<http://localhost:8000/dev-ui>**

- Select **`krakow_cultural_agent`**
- Inspect the live agent graph
- Examine execution traces & tool calls

35 TESTS PASSING IN STEP 5



Live Demo & Interactive Scenarios

SCENARIO 1: LIVE SCHEDULE

"When is the next Hejnał Mariacki?"

- Agent dispatches `getTrumpetCallSchedule()`
- Returns minutes remaining and 4 directions
- Cites St. Mary's Basilica historical corpus

SCENARIO 2: TICKET AVAILABILITY

"Are there tickets for Wawel tomorrow?"

- Agent dispatches `getWawelTicketAvailability()`
- Simulates real-time inventory across exhibitions

SCENARIO 3: ZERO-DOWNTIME HOT RELOAD

Live Price Update Test

1. Ask: "What is the State Rooms ticket price?" (Agent says 35 PLN).
2. Edit `data/mutable/wawel_pricing.md` to 55 PLN and save.
3. Ask again: Agent immediately answers **55 PLN** without restart!

SCENARIO 4: OBSERVABILITY

ADK Dev-UI Inspection

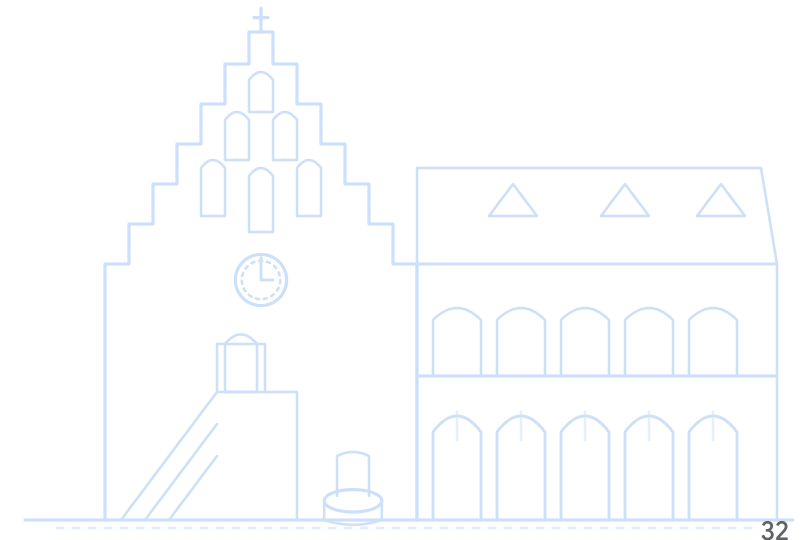
Trace the prompt, tool call payload, and model synthesis step-by-step.

● Step 06

Dynamic Cross-Content Grounding

Model Context Protocol (MCP) & Google Search Server

- Breaking the closed-world limitation of local-first RAG
- Standalone stdio MCP server using `@modelcontextprotocol/sdk`
- Dynamic Kraków events calendar & live Google Custom Search
- First-class integration via Google ADK `MCPToolset`



The Closed-World Problem in Local AI

Breaking Beyond Static RAG & Local Tool Boundaries

THE LOCAL-ONLY BOTTLENECK

Static Grounding Limits

- Static corpus only knows pre-indexed history
- Local mutable markdown requires manual authoring
- **What about:**
 - Annual film & music festivals (KFF, Jewish Culture)?
 - Temporary museum exhibits (*Lady with an Ermine*)?
 - Airport commuter rail updates & live weather?

THE SOLUTION

Model Context Protocol (MCP)

Open standard for dynamic tool discovery & execution over **stdio JSON-RPC**.

- Decouples search infrastructure from LLM logic
- Works with on-device Gemma 4 via Ollama
- First-class integration with **Google ADK**

Core Principle: When local RAG and deterministic tools don't have the answer, the agent autonomously queries the live world through an MCP Search Server.

The Tri-Layer Knowledge Architecture

Layered Grounding Strategy for Enterprise AI Agents

LAYER 1: STATIC RAG

Historical Corpus

Immutable monuments, architectural facts, and legends.

- Wawel, Basilica, Kazimierz
- In-memory BM25 retrieval
- **Latency:** < 1ms

LAYER 2: MUTABLE RAG

Live Markdown

Operator-editable filesystem data with timestamp caching.

- Pricing, hours, emergency lines
- Zero-downtime `mtime` reload
- **Latency:** < 2ms

LAYER 3: DYNAMIC MCP

Live Google Search

External cross-content and municipal calendar intelligence.

- Festivals, exhibitions, transit
- Stdio JSON-RPC protocol
- **Latency:** On-demand

DETERMINISTIC HIERARCHY: Local Grounding First → Domain Functions Second → External MCP Search Third.

Inside the Google Search MCP Server

Standalone Stdio Process with `@modelcontextprotocol/sdk`

Architecture & IPC Transport

```
// src/mcpServer.js
import { Server } from '@modelcontextprotocol/sdk/server/index.js';
import { StdioServerTransport } from '@modelcontextprotocol/sdk/server/stdio.js';
const server = new Server({
  name: 'google-search-mcp-server',
  version: '1.0.0'
}, { capabilities: { tools: {} } });

// stdio JSON-RPC transport
const transport = new StdioServerTransport();
await server.connect(transport);
```

Exposed MCP Tools

TOOL 1: GOOGLE_SEARCH

Queries Google Custom Search API or smart Kraków dynamic provider (festivals, museum loans, weather, transit).

TOOL 2: GET_KRAKOW_EVENTS_CALEDAR

Structured seasonal cultural events registry with category filters (`film`, `music`, `art`, `tradition`).

Binding MCP to Google ADK Agent

Seamless Integration via **MCPToolset** & **InMemoryRunner**

1. Connect MCPToolset via Stdio

```
// src/mcpClient.js
import { MCPToolset } from '@google/adk';
export function
createMcpToolset(serverPath) {
  return new MCPToolset({
    type: 'StdioConnectionParams',
    serverParams: {
      command: process.execPath,
      args: [serverPath]
    }
  });
}
```

2. Unify Native & MCP Tools

```
// src/agent.js
// Asynchronously discover MCP tools
const mcpTools = await mcpToolset.getTools();
// Build ADK agent with ALL tools
this.agent = new Agent({
  name: 'krakow_cultural_mcp_agent',
  model: this.ollamaLlm,
  instruction: systemPrompt,
  tools: [
    ...this.nativeTools,
    ...mcpTools // MCP tools seamlessly
    bound!
  ]
});
```

Hands-on Lab: Dynamic MCP Cross-Content

Step 06 Execution, Verification & Live Telemetry

Launch Step 06 Server

```
cd steps/step-06-google-search-mcp  
cp .env.example .env  
npm start
```

Open **<http://localhost:3030>**

42 TESTS PASSING IN STEP 6 (121 total across all steps)

TRY DYNAMIC QUERIES IN UI

1. **"When is the Jewish Culture Festival in Kazimierz?"**
→ Triggers MCP tool; grounds on summer festival schedule.
2. **"Where can I see Leonardo da Vinci's Lady with an Ermine?"**
→ Cites Czartoryski Museum temporary exhibit via MCP.
3. **"Fastest way from Kraków Airport to Old Town?"**
→ Retrieves SKA1 Balice rail link info via MCP.

Instructor Troubleshooting & Common Traps

Problem	Root Cause	Immediate Fix
<code>ECONNREFUSED</code> <code>127.0.0.1:11434</code>	Ollama daemon not running	Run <code>ollama serve</code> in background terminal.
<code>model '<name>' not found</code>	Weights missing	Run <code>ollama pull <name></code> or set <code>OLLAMA_MODEL</code> in <code>.env</code> .
<code>EADDRINUSE: :::3030</code>	Port 3030 collision	Run <code>lsof -ti:3030 xargs kill -9</code> or update <code>.env</code> .
<code>EADDRINUSE: :::8000</code>	Port 8000 collision	Set <code>ADK_PORT=8001</code> in <code>.env</code> .
Out of Memory (OOM)	System RAM constrained	Switch to lighter model: <code>OLLAMA_MODEL=gemma2:2b</code> .
Student stuck on code step	Syntax or typo	Guide them to jump directly to the next step folder!

Key Architectural Takeaways

1. LOCAL-FIRST AI

Privacy & Zero Cost

On-device Gemma 4 delivers fast, secure, deterministic AI without cloud dependency or token billing.

2. LEAN RAG

No Vector DB Bloat

In-memory lexical search (BM25 + Polish normalization + `mtime`) is faster and more accurate for domain tasks.

3. OFFICIAL GOOGLE ADK & MCP

Enterprise Architecture

Official `Agent`, `BaseLlm`, `FunctionTool`, and `MCPToolset` patterns scale cleanly to production.

Production Metric: 121 unit and integration tests passing in under 10 seconds across all 6 workshop steps with zero cloud dependencies.

 DevFest Kraków 2026 • Google Developer Experts

Dziękuję! / Thank You!

Questions & Community Discussion

- **Workshop Blog:** https://nja.dev/posts/devfest_2026_krakow
- **GitHub Repository:** <https://github.com/nuno-joao-andrade-dev/devfest-2026-krakow>
- **Plumar Agent CLI (Local LLMs):** <https://github.com/nuno-joao-andrade-dev/plumar>
- **DrawIt Architecture Diagramming:** <https://drawit.nja.dev/>
- **LinkedIn:** <https://www.linkedin.com/in/nuno-andrade-964b2416>
- **Documentation:** `WORKSHOP.md` & `SPEC.md`
- **Google ADK:** `@google/adk` & `@google/adk-devtools`
- **Google Open Models:** ai.google.dev/gemma

